

Java Programs – Explanations

Simple exam-oriented explanations of the programs discussed.

1. Using the this Keyword

Program:

```
class Demo {
    int id;
    String name;

    Demo(int id, String name) {
        this.id = id;
        this.name = name;
    }

    void display() {
        System.out.println("ID: " + this.id);
        System.out.println("Name: " + this.name);
    }

    public static void main(String[] args) {
        Demo d = new Demo(101, "Anu");
        d.display();
    }
}
```

Explanation:

- `this` refers to the current object.
- `this.id` is the instance variable and `id` is the constructor parameter.
- It is used when instance variables and parameters have the same name.

Exam point: `this` refers to the current object.

2. Constructor Chaining using this()

Program:

```
class Employee {
    int id;
    String name;

    Employee() {
        this(101, "Anu");
    }

    Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    void display() {
        System.out.println("ID: " + id);
        System.out.println("Name: " + name);
    }

    public static void main(String[] args) {
        Employee e = new Employee();
        e.display();
    }
}
```

Explanation:

- `this()` calls another constructor of the same class.
- `new Employee()` calls the no-argument constructor.
- `this(101, "Anu")` then calls the parameterized constructor.
- `this()` must be the first statement in a constructor.

Exam point: Calling one constructor from another constructor of the same class is called constructor chaining.

3. Garbage Collection

Program:

```
class Demo {  
    public static void main(String[] args) {  
        Demo d1 = new Demo();  
        Demo d2 = new Demo();  
  
        d1 = null;  
        d2 = null;  
  
        System.gc();  
  
        System.out.println("Garbage collection requested");  
    }  
}
```

Explanation:

- Java automatically manages memory using garbage collection.
- Objects that are no longer reachable become eligible for garbage collection.
- `d1 = null` and `d2 = null` remove these references.
- `System.gc()` requests garbage collection, but does not guarantee immediate execution.

Exam point: Garbage collection removes unreachable objects and helps reclaim memory automatically.

4. Stack Implementation using Array

Program:

```
class Stack {
    int[] stack = new int[5];
    int top = -1;

    void push(int value) {
        if (top == 4)
            System.out.println("Stack Overflow");
        else {
            top++;
            stack[top] = value;
        }
    }

    void pop() {
        if (top == -1)
            System.out.println("Stack Underflow");
        else
            top--;
    }

    void display() {
        if (top == -1)
            System.out.println("Stack is Empty");
        else {
            for (int i = top; i >= 0; i--)
                System.out.println(stack[i]);
        }
    }
}
```

Explanation:

- A stack follows LIFO: Last In, First Out.
- `top` stores the position of the top element and starts at `-1`.
- `push()` adds an element at the top.
- `pop()` removes the top element.
- Overflow occurs when the stack is full; underflow occurs when it is empty.
- `display()` prints elements from top to bottom.

Exam point: Stack operations are push, pop, and display, and the stack follows LIFO.

5. Returning an Object

Program:

```
class Book {
    int price;
    String title;

    Book(int price, String title) {
        this.price = price;
        this.title = title;
    }

    Book getBook() {
        Book b = new Book(500, "Java Programming");
        return b;
    }

    void display() {
        System.out.println("Price: " + price);
        System.out.println("Title: " + title);
    }

    public static void main(String[] args) {
        Book b1 = new Book(300, "C Programming");
        Book b2 = b1.getBook();
        b2.display();
    }
}
```

Explanation:

- A method can return an object.
- The return type of `getBook()` is `Book`, which is the class name.
- `return b` returns the reference to the created object.
- `Book b2 = b1.getBook()` stores the returned object reference in `b2`.

Exam point: A method can return an object by using the class name as its return type.

6. Private Access Specifier

Program:

```
class LibraryCard {
    private int cardNo;
    private String holderName;

    void setDetails(int c, String n) {
        cardNo = c;
        holderName = n;
    }

    void display() {
        System.out.println("Card Number: " + cardNo);
        System.out.println("Holder Name: " + holderName);
    }

    public static void main(String[] args) {
        LibraryCard l = new LibraryCard();
        l.setDetails(501, "Kiran");
        l.display();
    }
}
```

Explanation:

- `private` restricts direct access to a member from outside its class.
- `cardNo` and `holderName` can be accessed inside `LibraryCard`.
- `setDetails()` assigns values and `display()` reads them.
- Direct access such as `l.cardNo` from outside the class would cause an access error.
- Private members provide data hiding and support encapsulation.

Exam point: A private member can be accessed only within the class where it is declared.